

ALIBABA CLOUD × ATLAS AGENTIC AI HACKATHON

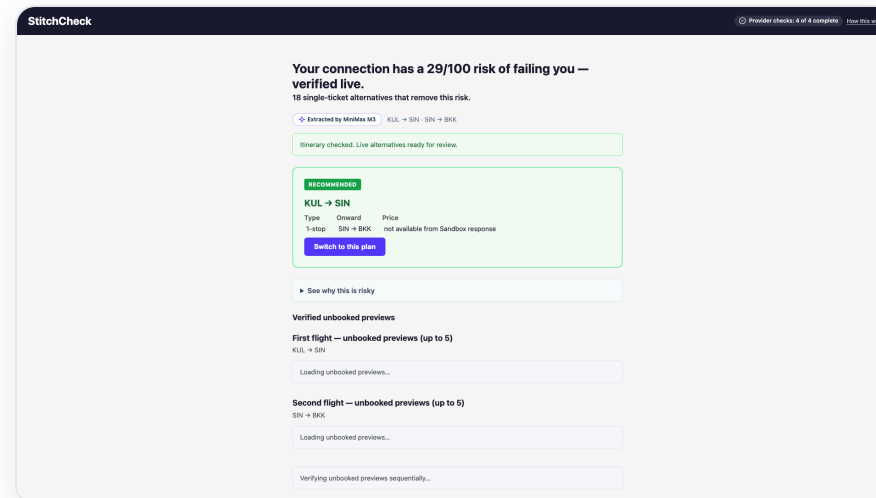
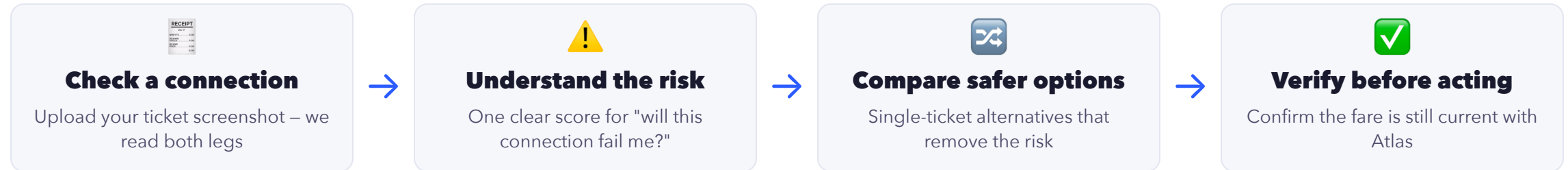
StitchCheck Itinerary Risk Checker

Check a connection, understand the risk, compare safer options, and verify before acting – then rehearse the full order-to-ticket journey **safely in the Atlas Sandbox.**

A tight connection can turn a cheap trip into a missed-flight cost

- **Two separate tickets, one fragile chain.** If the first flight runs late, the second ticket is the traveler's problem – no airline owes a rebooking.
- **The price you saw isn't the price you get.** Fares drift between searching and booking, sometimes within minutes.
- **The risk is invisible at checkout.** Booking sites show the cheapest option, never "will this connection fail you?"
- **The way out is one search away – but never shown.** Single-ticket alternatives that remove the risk entirely rarely appear at the moment of decision.

Four steps, one screen, before you commit



StitchCheck options screen: the risk headline and safer single-ticket alternatives, together on one screen.

Constraints first, trade-offs second, a human last

- **Hard constraints first.** Connection window, airport changes, viability threshold – violations are listed, never hidden.
- **Then trade-offs.** Price Δ vs arrival impact vs buffer minutes; missing data prints "not available", never fabricated.
- **Re-plans max twice.** And "no safe plan" is an answer it will show you.
- **Every write stops at a human.** 7 gates, all fail-closed.

Constraint ordering and fallback semantics per `core/domain/` and `core/safety/`; gate chain covered in detail on the Reliability & Safety slide.

Each service does one job – with visible evidence

ITINERARY EXTRACTION

MiniMax M3 via Nosana

Ticket screenshot → structured, editable itinerary – the in-app "Extracted by MiniMax M3" tag is state-derived.

Live, verified

RISK COMPUTATION

Nosana

Monte Carlo connection-risk simulation on decentralized GPU – one completed job, IPFS-anchored evidence.

Live, verified

SANDBOXED SCORING

Daytona

Reproducible risk-scoring worker in an isolated, network-blocked sandbox – shown in-app under "How this was calculated".

Live, verified

FARES & ALTERNATIVES

Atlas Sandbox • live

Real flight search and price verification behind a read-only gate – no booking writes on the verify path.

Live, verified

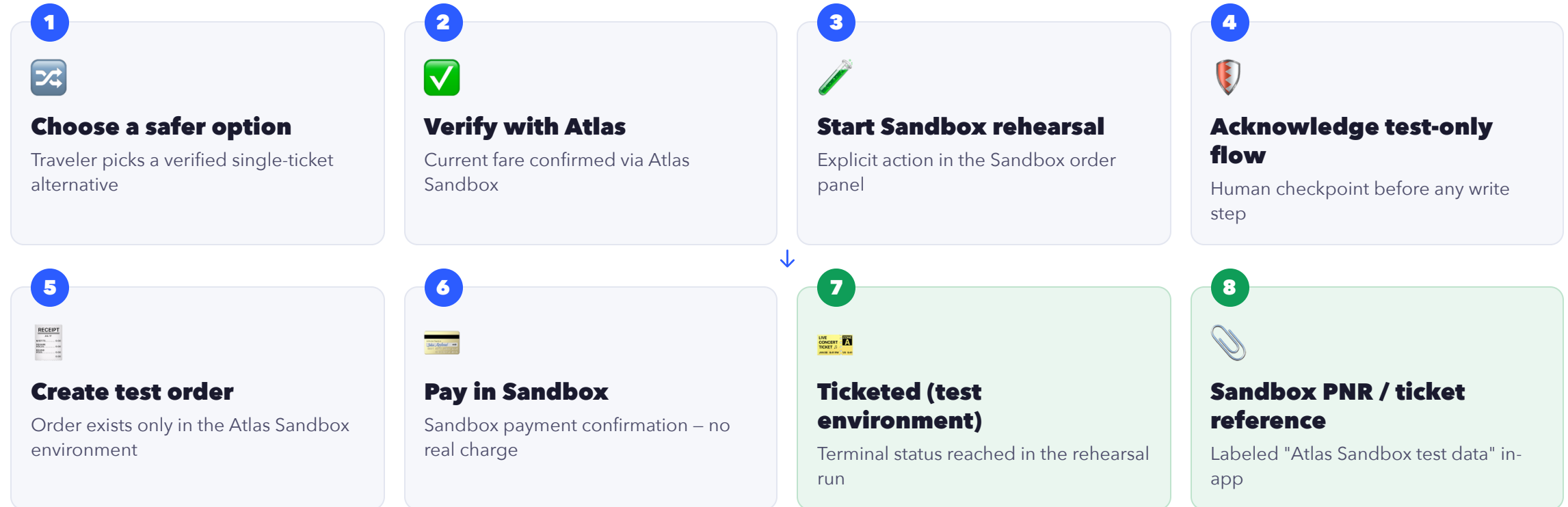
WHERE IT RUNS **Alibaba Cloud • staged**

Staged – not deployed yet

Local dev today → staged for Alibaba Cloud (ap-southeast-1, Singapore): SAE container from our fail-closed Dockerfile · OSS private evidence store (SSE-KMS) · KMS-injected secrets – no key ever in image or git. Security: VPC-isolated, egress allow-listed to sponsor APIs; risk compute stays in Daytona's network-blocked sandbox. Envelope: < US\$16/mo demo scale · measured 6 ms risk compute · \$0.044 Monte-Carlo job.

Evidence per README "Submission evidence status": `demo-evidence/2026-08-29-submission-final/` · `smoke-tests/nosana/results/` · `smoke-tests/daytona/results/` · `smoke-tests/atlas/results/` · deployment path per `deploy/README.md`

From safer option to Ticketed – safely in Atlas Sandbox



Demo mode – no real booking, charge, or airline ticket is created.

Flow diagram (not an app screenshot). Successful rehearsal recorded in `output/atlas-sandbox-evidence-2026-08-29.jsonl` and README: order `TESTA20260829181717829`, PNR `S30798`, terminal status `TICKETED` – Atlas Sandbox only.

Forgiving on flakiness, strict on anything that writes

- **One-click Verify, resilient underneath.** Search and Verify retry transient provider availability issues (e.g. `SERVICE_TEMPORARILY_UNAVAILABLE`) automatically while the UI is loading – no silent dead ends.
- **Fail-closed safety gates protect order, payment, and ticketing steps.** A 7-layer gate chain – all seven must pass before any sandbox write executes; every flag defaults to off:

Ticketing availability

Authorization

Route requirements

Sandbox mode

Sandbox base-URL exact match

Payment confirmation

Ticketing confirmation

Verification

- **Production and sandbox writes are mutually exclusive.** The production write kill switch stays off – enforced and covered by the offline gate test suite.
- **No real payment or ticket issuance.** Everything past Verify happens in the Atlas Sandbox test environment.

Per README safety architecture (7-layer gate chain) and `app/src/atlas/transient-retry.ts` (docs/session-handoff-2026-08-30-transient-retry-commit.md).

PROOF OF THE END-TO-END JOURNEY

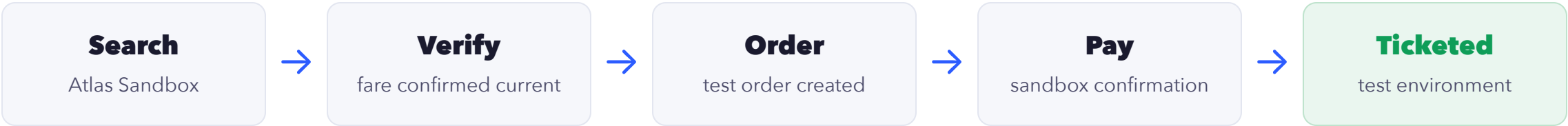
Search → Verify → Order → Pay → Ticketed

The full pipeline was executed end-to-end against the Atlas Sandbox – test environment only.

✓ **Completion state – Ticketed (test environment)**

Sandbox rehearsal finished with terminal status `TICKETED` and a `Sandbox PNR / ticket reference` (Atlas Sandbox test data).

Recorded in `output/atlas-sandbox-evidence-2026-08-29.jsonl` · order `TESTA20260829181717829` · PNR `S30798` (README submission evidence).



Completion-state diagram (not an app screenshot). Pay steps with transient timeouts were re-driven to accepted `TICKETED` outcomes – see the 2026-08-29 / 2026-08-30 evidence logs.

Spec → Agent → Expert → offline tests – before any merge



- **The full build trail is auditable.** Every agent prompt brief is kept in-repo under `a/` – nothing was summarized after the fact.
- **Reproduce it yourself:** `node scripts/qoder-build-evidence.mjs` – prints spec, brief, LOC, and test-suite counts from this repo.

Counts verified 2026-08-30 by the script above; workflow documented in `docs/qoder-implementation-evidence.md`.

WHY THIS MATTERS

An unpriced, unprotected behavior at APAC scale

- **A multi-billion behavior, zero contractual protection.** Self-transfer stitching is how APAC LCC travellers buy – the traveller carries 100% of the risk.
- **One misconnection costs US\$150-400 to rebook.** And we watched a "current" fare swing +217% (\$64.38 → \$203.99) in one search→verify cycle.
- **Every risk warning is a monetizable switch moment:** a verified single ticket, offered at booking-intent peak.
- **Model:** per-booking fee on converted alternatives (OTA/TMC) · per-check SaaS (insurers).

+217% fare swing is our own measured evidence: README "Submission evidence status" · fare `off_11db11bad81302c295da16f1`. Rebooking-cost range is an industry estimate, not our measurement.

CLOSING · WHERE TO VERIFY EVERYTHING

StitchCheck makes risky connections understandable before they become expensive.

- **Code:** `github.com/yahgoo/StitchCheck` → branch `feature/atlas-sandbox-mock-ticketing` (QR placeholder – point at the repo branch, not a demo URL)
- **Deck:** [StitchCheck-Alibaba-Atlas-Hackathon-2026-Updated.pdf](#) (11 slides, Google Drive)
- **Demo video:** local artifact, recording now – will be linked from the README "Demo video" line
- **One command to verify everything offline:** `npm --prefix app run verify:offline`

Try it: upload a ticket screenshot → see the risk → choose the safer way